Peanut SDKV1.3.0 Development Documentation

I. Overview

Adapt to a variety of models and configurations, open up the underlying capabilities of the robot, encapsulate the functional components, and simplify the realization of the scene.

II. SDK Integration Guide

1. Introduction

This document is intended to guide developers in developing Android applications running on Peanut robots using the SDK.

- Please read carefully before integration Preparation to help you understand what you need to do before integrating.
- If you want to learn how to run the sample project Peanut Sample, check out the Sample Run for more information on how to run the sample project, Peanut Sample.
- If you want to know how to integrate the SDK into your own application, please check out the SDK Integration for more information on how to integrate SDKs into your application.
- If you want to learn about simple integration of the SDK code for use, check out the SDK Initialization If you want to know more about the methods in the SDK, please check the "API List".
  in the SDK in more detail, please check the API List.
- If an error is encountered during integration development, please refer to Constant Field Values - Error Codes.

2. Preparatory work

- Prepare a Peanut robot.
- Prepare a male-to-male dual-ended USB cable, recommended 2~3 meters.

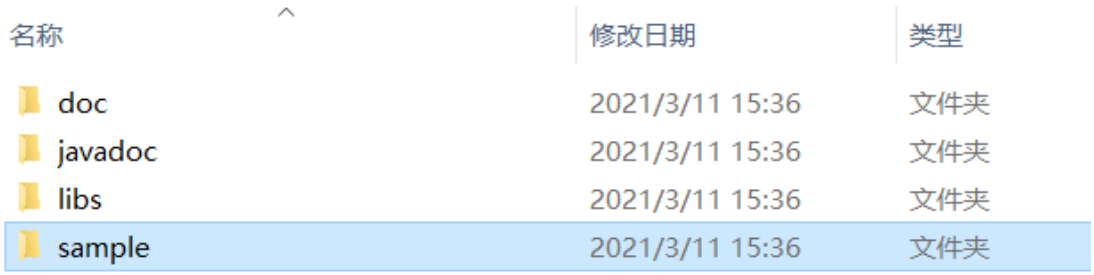
- Contact technical support to set up the on-site robot operating environment and obtain a copy of the Robot Operating Target Point Location Table from technical support.
- Contact Sales to request offline forensic information.

3. Integration process

Developers need to apply for development related resources from the platform, including Android SDK, samples, development documents, technical support, etc.

3.1. Getting the sdk

Get the SDK distribution file, the SDK file is usually a zip file named as peanut-sdk-x.x.x.zip, where x.x.x represents the SDK version, whichever is the actual version. After decompression, you can get the following file directory:



The file directory is described below:

- doc is the directory where this document is located.
- javadoc is.

- libs are SDK library files.
- sample is a demo sample program containing the basic functions of the SDK, which can be opened and compiled directly with Android Studio.

3.2. Sample operation

3.2.1. Step 1: Robot power-up

Turn on the robot power and wait for the robot to finish powering up.

3.2.2. Step 2: Preparing the robot software runtime environment

- If the robot boots up and goes directly to the robot's default application, contact technical support for instructions on how to exit the self-starting program and cancel the robot's default application's power-up and process guarding.
- Check and force stop the default application. To do this, go back to the Android desktop, click [Settings], [Apps], select the default Peanut program, and click [Force Stop]. This step is to prevent the default application and Peanut Sample program from communicating with the robot internally at the same time causing communication errors.

3.2.3. Step 3: Prepare the Robot Hardware Commissioning Environment

- Open the back part of the robot's waist belt, open the method is to press the two sides of the buckle and pop up, force to break down the center buckle, remove the belt can be seen on the right side of the USB port. This USB port can be set to HOST mode or DEVICE mode. When set to HOST mode, a USB flash drive can be used. When set to DEVICE mode, you can use ADB to debug Android applications.

- Set USB to DEVICE mode. The specific steps are, go back to Android desktop, tap out [Settings], [Developer Options], [USB Mode Switching], and check [Device Mode].

- Use a dual-ended USB cable to connect the PC to the USB port on the waist of the robot (location varies by model, consult technical support).



3.2.4. Step 4: Connect the PC debugging environment and run it

- Open Android Studio. sample program is developed in Android Studio and tested.

- Open the Sample project in the SDK files directory.

- Compile and run. the Sample program runs on the robot.

3.3. SDK integration

3.3.1 Adding library files

Copy the aar file from the libs directory in the SDK directory to the libs directory of the Android project module.

3.3.2. Configuring build.gradle

- android Configuring packagingOptions

```
packagingOptions {
    pickFirst 'about.html'
    pickFirst 'edl-v10.html'
    pickFirst 'epl-v10.html'
    pickFirst 'notice.html'
    pickFirst 'META-INF/legal/LICENSE'
    pickFirst 'META-INF/legal/NOTICE.md'
    pickFirst 'META-INF/legal/3rd-party/cc0-legalcode.html'
    pickFirst 'META-INF/legal/3rd-party/BSD-3-Clause-LICENSE.txt'
    pickFirst 'META-INF/legal/3rd-party/APACHE-LICENSE-2.0.txt'
    pickFirst 'META-INF/legal/3rd-party/MIT-license.html'
    pickFirst 'META-INF/legal/3rd-party/CDDL+GPL-1.1.txt'
}
```

- Ensure dependency on libs/peanut-sdk-xxx.aar:

```
dependencies {
    implementation fileTree(dir: 'libs', include: ['*.aar'])
    implementation 'com.alibaba:fastjson:1.2.68'
    implementation 'com.google.code.gson:gson:2.8.6'
    implementation 'org.slf4j:slf4j-api:1.7.25'
    ...
}
```

3.3.3. Configuring AndroidManifest.xml

Configure the permissions required by the SDK

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.READ_PHONE_STATE" />
```

At this point, developers can call the functionality provided by the Peanut SDK in their own applications.

3.4. SDK initialization

3.4.1.

```
PeanutSDK.getInstance().init(this.getApplicationContext(), mErrorListener);
```

3.4.2 Release

When the application exits, the release of the SDK needs to be called to avoid resource utilization.

```
PeanutSDK.getInstance().release();
```

3.4.3 Detailed configuration

```
PeanutConfig.getConfig()
   //Set the connection protocol
    .setLinkType(PeanutConstants.LinkType.COM_COAP);
    //Setup connection to motherboard serial port
    .setLinkCOM(PeanutConstants.COM1);
    //Set the connection service IP
    .setLinkIP(PeanutConstants.LOCAL_LINK_PROXY); .
    //Set the connection service port
    .setLinkPort(5683); .
    //Setup Lamp Board Serial Port
    .setEmotionLinkCOM(PeanutConstants.COM2);
    //Set up the gated board serial port
    .setDoorLinkCOM(PeanutConstants.COM2);
    //Set the number of doors
    .setDoorNum(1);
    //Set the connection timeout
    .setConnectionTimeout(PeanutConstants.COAP_TIME_OUT);
```

```
    //Whether to enable logging
    .enableLog(true)
    //Set the log level
    .setLogLevel(Log.DEBUG)
    //Set AppId for offline authentication
    .setAppId("xx")
    //Set Secret for offline authentication
.setSecret("xx");
PeanutSDK.getInstance().init(this.getApplicationContext(), config, mErrorListener);

Error code for callback after //SDK initialization, 0 for success, other fault
conditions (refer to Constant Field Value - Error Code)
private PeanutSDK.InitListener mErrorListener = new PeanutSDK.ErrorListener() {

    @Override
    public void onInit(int errorCode) {
        Log.d(TAG, "onInit:" + errorCode);
    }
};
```

3.5 Description of parameters

3.5.1 Communication protocols

RobotSDKConfig allows you to set the communication protocol, machine IP, port, serial port and so on.

```
public enum LinkType {
```

```
    DEFAULT.
    COM.
    COM_COAP.
    COAP.
    HTTP
}
```

Agreed Value	protocol description
DEFAULT	Reserved fields
COM	serial port protocol
COM_COAP	Coap Agent, Serial Protocol
COAP	Coap Agreement
HTTP	Http protocol

3.5.2 Serial ports

```
  public static final String COM1 = "/dev/ttyS1";
  public static final String COM2 = "/dev/ttyS2";
```

3.5.3 Address for communication services

```
  public static final String LOCAL_LINK_PROXY = "127.0.0.1";
  public static final String REMOTE_LINK_PROXY = "192.168.64.20";
```

Service IP	applicable conditions
LOCAL_LINK_PROXY	stereoscopic visual orientation
REMOTE_LINK_PROXY	LiDAR positioning

4. Help

If you encounter integration problems, you can first refer to the latest version of the documentation and Sample examples.

When you encounter environment configuration can not be resolved, please communicate with technical support first to confirm whether the deployment is correct.

Please follow the problem feedback template to contact technical if you have problems using the feature.

III. Description of components

1. Equipment runtime components

1.1 Introduction to utilization

The component is used for device connection, and after connection, it performs self-test, synchronization of the state of each component of the device, synchronization of parameters, acquisition and synchronization of device information, and filters out unreasonable data and unchanged subscription reporting.

```
public class RuntimeInfo {
  //Operating mode
  private int workMode.

  //Synchronization status
  private int syncStatus.

  //Power
  private int power.

  //Total mileage
  private Double totalOdo.

  // Emergency Stop Enable
  private boolean emergencyEnable.

  // Emergency stop switch
  private boolean emergencyOpen.

  //Motor status
```

```
    private int motorStatus.

    //Algorithm board information
    private String robotArmInfo.

    //Motion Board Information
    private String robotStm32Info;

    //robot IP address
    private String robotIp.

    //Parameter Configuration
    private String robotProperties.

    //All target points
    private String destList;
}
    PeanutRuntime.getInstance().start(new PeanutRuntime.Listener() {
      @Override
      public void onEvent(int event, Object obj) {
        Log.d(TAG, "onEvent: " + event + ", content: " + obj);
      }

      @Override
      public void onHealth(Object content) {
        Log.d(TAG, "onHealth:" + content);
      }

      @Override
      public void onHeartbeat(Object content) {
        Log.d(TAG, "onHeartbeat:" + content);
      }
```

```
    }).
```

1.2 Functional APIs

Qualifiers and types	Methodology and instructions	Associative Constants
static PeanutRuntime	getInstance() Get the single instance.	
RuntimeInfo	getRuntimeInfo() Get runtime information, including operating mode, power, mileage, version information, IP, model configuration, emergency stop button status, motor status, etc.	
void	start() Starts, establishes a communication connection, begins data subscription, configuration synchronization, state self-tests, etc.	
void	registerListener(Listener listener) Register callbacks to synchronize event and status information.	
void	removeListener(Listener listener) Write-off callbacks.	
void	setEmergencyEnable(final boolean enable) Sets whether the emergency stop button is enabled or not.	
void	setIP(String ip, int port) Set the service address for multi-computer collaboration scenarios.	

void	setWorkMode(final int mode) Set the operating mode.	Constants - Operating Modes
void	setTime(final long timestamp) Setting robot time	
void	syncParams2Robot(boolean needReboot) Synchronize the model configuration, whether or not a reboot takes effect.	
void	location() Power-on localization for SLAM localization.	
void	getPath() Get location route information	
void	destory() Destruction.	

1.3. Listener

Qualifiers and types	Methodology and instructions	Associative Constants
void	onEvent(int event, Object obj) Returns the event code and related information data.	Events - Device Operation
void	onHealth(Object content) Healthy state callbacks.	
void	onHeartbeat(Object content) Heartbeat state callback.	

```
public interface Listener {
  void onEvent(int event, Object obj).

  void onHealth(Object content);

  void onHeartbeat(Object content).
```

```
}
```

2. navigation component

2.1 Introduction to utilization

The component is used for robot autonomous navigation control. It can set the route planning strategy, arrival strategy, destination list, number of cycles, etc. After starting navigation, it can control the pause, resume and stop of navigation, and it can synchronize and report the navigation status and navigation information during navigation.

2.2 Functional APIs

2.2.1 PeanutNavigation.Builder class

Qualifiers and types	Methodology and instructions	note
pulic	Builder() Construction Methods.	
Builder	setTargets(Integer... targets) Set the list of destinations, integer.	
Builder	enableDefaultArrival(boolean enable) Enable the default arrival policy.	
Builder	setRoutePolicy(int policy) Set up a route planning strategy.	Constants - Route Planning Strategies
Builder	enableAutoRepeat(boolean autoRepeat)	

	Enable Auto Loop.	
Builder	setRepeatCount(int count) Set the number of cycles.	Target Point Navigation Task Cycle
Builder	setArrivalControl(ArrivalControl arrivalControl) Set up a customized arrival policy.	
Builder	setArrivalControl(boolean enable, float marginScope, int blockDelay, int scopeDelay) Set the default arrival policy.	
Builder	setRouteSelector(RouteSelector routeSelector) Setting up a customized route planning strategy	
Builder	setListener(Navigation.Listener listener) Setting Callbacks	
Builder	setBlockingTimeOut(int timeout) Set blocking timeout, default arrival after timeout, switch to next task, unit ms	Laser positioning only
PeanutNavigation	build() Creating Methods	

2.2.2. PeanutNavigation API

Qualifiers and types	Methodology and instructions	Associative Constants

pulic	PeanutNavigation(ArrivalControl arrivalControl, RouteLine routeLine, Navigation.Listener listener, int blockingTimeout) Construction Methods.	
void	setTargets(Integer... targets) Set the target point list, the parameter is a list of target point IDs, the IDs come from the map.	
void	setTargets(List<RouteNode> targets) Sets the list of destination points with the RouteNode list parameter.	
void	prepare() Preparation, route planning.	
void	setPilotWhenReady(boolean pilotWhenReady) Autonomous navigation, control start and pause when ready.	
void	setSpeed(int speed) Set the navigation motion speed in cm/s.	
void	pilotNext() Switch to the next target point and start navigation.	
void	stop() Stop the navigation.	
void	release() Release of resources.	
void	manual(int direction) Manual navigation.	Constant - manual navigation direction
RouteNode[]	getRouteNodes()	

	Get a list of current target points.	
RouteNode	getCurrentNode() Get the current navigation target point.	
RouteNode	getNextNode() Get the next navigation target point.	
boolean	isLastNode() Whether or not the last navigation target point.	
int	getCurrentPosition() Gets the index of the current navigation target point.	
void	skipTo(@NonNull int node) The jumping knife designates the point at which the lower target is located.	
void	cancelArrivalControl() The current point navigation cancels the proximity reach.	
void	setArrivalControlEnable(boolean enable) Sets whether proximity arrivals are enabled.	
boolean	isLastRepeat() Whether it is the last cycle	
void	addListener(Navigation.Listener listener) Adding Callbacks	
void	removeListerner(Navigation.Listener listener) Remove Callbacks	

2.3 Customized strategies

2.3.1 Arrival strategy

Developers implement the ArrivalControl interface according to their own needs. Call the onArrived() method of the Listener after reaching the determination.

```
public interface ArrivalControl {
  public interface Listener {
    void onArrived().
  }

  public static final float DEFAULT_MARGIN_SCOPE = 1.0f;
  public static final int DEFAULT_BLOCK_DELAY = 5 * 1000;
  public static final int DEFAULT_SCOPE_DELAY = 10 * 1000.

  public void addListener(Listener listener);

  public void removeListener(Listener listener).

  public void enable(boolean enable).

  public void notifyDistanceChanged(float distance).

  public void notifyBlocked();

  public void release();
}
```

2.3.2 Route planning

Developers implement their own route planning according to their needs.

```
public interface RouteSelector {
  interface Output {
    void adaptiveRoute(RouteNode... routeNodes).
```

```
    void fixedRoute(RouteNode... routeNodes).

    void onError(int errorCode).
  }

  public static final int DEFAULT_LIMIT = 5;

  public void selectRoute(Output output, RouteNode... RouteNodes).
}
```

The maximum limit for single path planning is 5.

2.4. Listener

Qualifiers and types	Methodology and instructions	Associative Constants
void	onStateChanged(int state , int schedule) Returns the navigation status code. Parameters: state: the navigation state. schedule: whether scheduling is in progress.	Constants - Navigation Status
void	onRouteNode(int index, RouteNode routeNode) Target point update.	
void	onRoutePrepared(RouteNode... routeNodes) The route is ready.	
void	onDistanceChanged(float distance) The distance left to the current target point.	

void	onError(int code) Failure.	error code

```
public interface Navigation {

 final class Factory {
   private Factory() {
   }

   public static Navigation newInstance(ArrivalControl arrivalControl) {
     return NavigationImpl.getInstance(arrivalControl);
   }
 }

 public interface Listener {
   void onStateChanged(int state, int schedule).

   void onRouteNode(int index, RouteNode routeNode).

   void onRoutePrepared(RouteNode... routeNodes).

   void onDistanceChanged(float distance).

   void onError(int code).
 }

 public static final int STATE_IDLE = 0;;
 public static final int STATE_PREPARED = 1;
 public static final int STATE_RUNNING = 2;
 public static final int STATE_DESTINATION = 3;
 public static final int STATE_PAUSED = 4;
 public static final int STATE_COLLISION = 5;
 public static final int STATE_BLOCKED = 6;
```

```
public static final int STATE_STOPPED = 7;
public static final int STATE_ERROR = 8;
public static final int STATE_BLOCKING = 9;
public static final int STATE_END = 10;

public static final int FORWARD = 1;
public static final int BACKWARD = 2;
public static final int LEFT = 3;
public static final int RIGHT = 4;



public void addListener(Listener listener);

public void removeListener(Listener listener).

public void prepare(RouteLine routeLine);

public void setPilotWhenReady(boolean pilotWhenReady);

public void pilotNext();

public void stop();

public void manual(int direction).

public void setSpeed(int speed).

public void setBlockTimeout(int timeout);

public RouteNode[] getRouteNodes();

public RouteNode getCurrentNode();
```

```
    public RouteNode getNextNode();

    public int getCurrentPosition();

    public void release();

    public void skipTo(int index).

    public void cancelArriveControl();

    void resetNewTargets().

    boolean isLastRepeat();

    void arrivalControlEnable(boolean enable).
}
```

3. power assembly

3.1 Introduction to utilization

Component for robot charging control and listening to the power level. The robot can be set up for automatic charging, manual charging, and adapter charging. The robot will report charging events and power changes synchronously.

```
PeanutCharger mPeanutCharger.

//Charge callbacks
  Charger.Listener listener=new Charger.Listener() {
    @Override
    public void onChargerInfoChanged(int event, ChargerInfo chargerInfo) {
      PrintLnLog.d(ChargerDemo2.this, tvApiLog, svApiLog, sb, "event = " + event+
              " Power = " + chargerInfo.getPower()+" ChargeEvent = " +
chargerInfo.getEvent());
    }

    @Override
    public void onChargerStatusChanged(int status) {
      PrintLnLog.d(ChargerDemo2.this, tvApiLog, svApiLog, sb, "status = " +
status);
    }

    @Override
    public void onError(int errorCode) {
```

```
      PrintLnLog.d(ChargerDemo2.this, tvApiLog, svApiLog, sb, "errorCode = " +
errorCode);
    }
  };



 //Initialization
 mPeanutCharger = new PeanutCharger.Builder()
     .setListener(listener)
     .build();
 mPeanutCharger.execute();

//Charge command control
  case R.id.btn_manual_charge.
    mPeanutCharger.performAction(PeanutCharger.CHARGE_ACTION_MANUAL);
    break;
  case R.id.btn_auto_charge.
    mPeanutCharger.performAction(PeanutCharger.CHARGE_ACTION_AUTO);
    break;
  case R.id.btn_stop_charge.
    mPeanutCharger.performAction(PeanutCharger.CHARGE_ACTION_STOP);
    break;
  case R.id.btn_adapter_charge.
    mPeanutCharger.performAction(PeanutCharger.CHARGE_ACTION_ADAPTER);
    break;



//Release
// No need to release under normal conditions, built-in subscription power, can't
get power after release
if (mPeanutCharger ! = null) {
```

```
    mPeanutCharger.cancel();
}
```

3.2 Functional APIs

3.2.1 The Builder class

Qualifiers and types	Methodology and instructions	Associative Constants
public	Builder() Construction Methods.	
Builder	setPile(int pile) Setup Charging Post Id.	
Builder	setListener(Charger.Listener listener) Setting callbacks.	
PeanutCharger	build() Create method.	

3.2.2 API

Qualifiers and types	Methodology and instructions	Associative Constants
public	PeanutCharger(int pile, Charger.Listener listener,,int maxRetry,long timeRetry) Construction Methods.	

void	setPile(int pile) Setting up charging posts.	
void	execute() Beginning of implementation.	
void	performAction(int action) Control charging.	Constant - Power Charge Command
void	release() Release of resources.	
int	getPile() Get the electric stake ID.	

3.3. Listener

Qualifiers and types	Methodology and instructions	Associative Constants
void	onChargerInfoChanged(int event, ChargerInfo chargerInfo) Returns power supply reported events and power information.	Event - Power Charge Event
void	void onChargerStatusChanged(int status). Returns to the current state when the charge state changes.	Constant - Power Status
void	onError(int errorCode) Failure.	error code

```
public interface Listener {
```

```
    void onChargerInfoChanged(int event, ChargerInfo chargerInfo).

    void onChargerStatusChanged(int status).

    void onError(int errorCode).
  }

public class ChargerInfo implements Serializable {
  int power.
  int event.

  public int getPower() {
    return power.
  }

  public int getEvent() {
    return event.
  }

  public void setPower(int power) {
    this.power = power;
  }

  public void setEvent(int event) {
    this.event = event;
  }


  @Override
  public String toString() {
    return "ChargerInfo{" +
            "power=" + power +
```

```
            ", event=" + event +
            '}';
  }
}
```

4. audio component

4.1. Introduction to use

The component acquires audio streaming data through the T8 device microphone hardware.

```
// Step1 Getting the audio component
AudioComponent component = PeanutSDK.getInstance().audio();
// Step2 Initialize the audio component
Component.initAudio(mAudioListener);

private OnAudioListener mAudioListener = new OnAudioListener() {
    @Override
    public void onSuccess() {
      // Successful initialization, you can do some business logic processing.
    }

    @Override
    public void onError(int errorCode) {
      // Initialization failure: -100 Microphone hardware does not exist, -200
Microphone hardware open failure
    }
```

```
    @Override
    public void onAudioData(byte[] bytes, int len) {
      // Audio data callbacks
    }

    @Override
    public void onHeartbeat(int state) {}
};
```

4.2. Functional Api

Qualifiers and types	Methodology and instructions	Associative Constants
Int	audioStart() Started audio data reporting, 0 executed successfully; Reported upwards via the OnAudioListener.onAudioData() method.	
Int	audioStop() Stop audio data reporting, 0 Execution was successful.	
boolean	isInitialized() Whether or not it has been initialized.	
void	setAudioGainEnable(boolean enable) Audio gain switch, default off.	
void	setAudioGainValue(float value) Audio gain value setting.	
byte[]	gainAudioData(byte[] data) Audio gain data conversion.	

4.3. Listener

Qualifiers and types	Methodology and instructions	Associative Constants
void	onSuccess() Microphone pickup initialized successfully.	
void	onError(int errorCode) Microphone pickup initialization failed.	ERROR_DEVICE_NOT_FOUND error_device_open_failed
void	onAudioData(byte[] bytes, int len) Microphone Audio Data Callback	
void	onHeartbeat(int state) Microphone pickup heartbeat	

```
  // Microphone audio callback interface
public interface OnAudioListener {
         // Microphone hardware not present
         int ERROR_DEVICE_NOT_FOUND = -100;
         // Microphone hardware open failure
         int ERROR_DEVICE_OPEN_FAILED = -200;

         void onSuccess().
         void onError(int errorCode).
         void onAudioData(byte[] bytes, int len).
         void onHeartbeat(int state).
}
```

5. Hatch assemblies

5.1 Introduction to Basic Functions

5.1.1 Initialization before use

```
PeanutDoor.getInstance().init(getApplicationContext());


// Resources need to be released at the end of use:
PeanutDoor.getInstance().release() ;
```

5.1.2 Listening status

```
public interface DoorListener {
//return a certain ID hatch error doorId is null for a non-specified door error
  void onFault(Faults type, int doorId).
//Listening to the opening and closing status of specific ID hatches
  void onStateChange(int floorId, int state);
//Monitoring hatch type change
  void onTypeChange(GatingType gatingType).
//Listening for successful hatch setup
  void onTypeSetting(boolean success).
//Error code
  void onError(int errorCode).
}
```

5.1.3 Hatch control & hatch type description

Based on the current structural design, there are four types: four-compartment, two-compartment, T-compartment, and inverted T-compartment.

```
mPeanutDoor.setDoorType(Door.SET_TYPE_FOUR, TAG, this);
```

Constant - hatch setting type

numerical value	clarification
SET_TYPE_FOUR(101)	Set to four compartments
SET_TYPE_DOUBLE(102)	Set up as a double cabin
SET_TYPE_THREE(103)	Set to T-pod
SET_TYPE_THREE_REV(104)	Set to inverted T-pod
SET_TYPE_AUTO(105)	Set to automatic recognition

Hatch on/off control:

```
//single true : opens the specified hatch alone (all others are closed) false : does not affect each other
  public void openDoor(int doorId, boolean single) {
    mPeanutDoor.openDoor(doorId, single);
  }

 //Close specified hatch
  public void closeDoor(int doorId) {
    mPeanutDoor.closeDoor(doorId);
  }
```

5.1.4 Other functions

```
  // Close all hatches
  public void closeAllDoor() {
    mPeanutDoor.closeAllDoor();
  }
   // Determine if all hatches are closed
  public boolean isAllDoorClose() {
    return mPeanutDoor.isAllDoorClose();
  }
```

```
// Get the hatch firmware version number
 public String getDoorVersion() {
   return mPeanutDoor.getDoorVersion();
 }
 //Determine whether to support the function of manually changing the hatch type
 public boolean supportDoorTypeSetting() {
   return mPeanutDoor.supportDoorTypeSetting();
 }
```

5.2 API Summary

api	clarification
void setDoorListerner(String tag, DoorListener gatingFaultListener)	Setting Up Listening
void removeFloorListener(String tag).	Remove listener
int getDoorType();	Getting the hatch type
GatingState getDoorState(int doorId).	Get the status of the specified id hatch
void openDoor(int doorId, boolean single).	Controlling the opening of doors
void closeDoor(int doorId).	Control of closing doors
void closeAllDoor();	Control closure of all hatches
boolean isAllDoorClose();	Determining if multiple throttles are all closed
void setDoorType(int gatingType, String tag, DoorListener doorListener);;	Setting the hatch type
String getDoorVersion();; String getDoorVersion().	Get the hatch control version

boolean supportDoorTypeSetting();	Determine if the hardware supports the hatch type change function
void release().	Release of resources

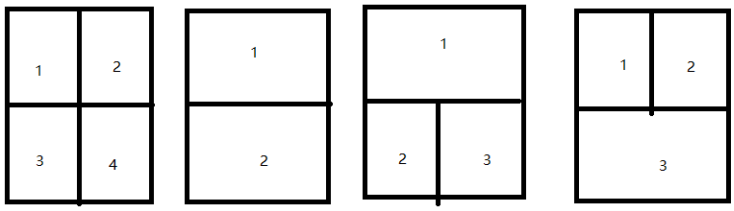
Additional Notes:

gatingType :

GatingType(enum)	TypeId (int)	typology
Four	0	quadruple cabin
DOUBLE	1	double cabin
THREE	2	T-shape
THREE_REVERSE	3	inverted T-shape

doorId:

According to the rule of left-to-right and top-to-bottom order, starting from 1 and increasing sequentially



5.3 Precautions

Before using the function of setting the hatch type, you need to determine whether the hatch firmware supports the hatch type setting function, otherwise the setting may be unsuccessful or unresponsive. This can be determined by the `supportDoorTypeSetting()` method.

6. Ros map file manipulation components

1.1 Introduction to utilization

The component is used for importing and exporting map files with the robot Ros system. After exporting the map from machine A, the exported PeanutDB.zip file is copied to the Sdcard directory of machine B with a USB flash drive to perform the import operation.

```
// Importing maps
MapManager.getInstance().onImportToRos();
// Export maps
MapManager.getInstance().onExportToAndroid();
```

1.2 Functional APIs

1.2.1. Builder class

Qualifiers and types	Methodology and instructions
public	addListen(MapListen listen) add callbacks

public	onImportToRos() Import map to Ros //default reads root files
public	onExportToAndroid() export map to android Sdcard //default root directory
public	onImportToRos(String filePath) Import map to Ros //specify directory files
public	onExportToAndroid(String savePath) Export map //Specify the directory of the exported file

1.3. Listener

Qualifiers and types	Methodology and instructions
void	onResult(int code)

1.3.1 Callback states

status code	clarification
1001	Imported map successfully
1002	Successful map export
2001	Failed to import map
2002	Failed to export map

6. T3 Hatch control

1.1 Introduction to utilization

Controls the T3 robot hatch opening and closing operation.

```
// Open the designated hatch
SensorDoor.getInstance().setDoorSwitch(doorID,true)
//Close specified hatch
SensorDoor.getInstance().setDoorSwitch(doorID,false )
```

1.2 Functional APIs

1.2.1. hatch control

Qualifiers and types	Methodology and instructions
public	SensorDoor.getInstance().setDoorSwitch(doorID,isOpen)

1.2.2 Hatch ID

ID	
ProtoDev.SENSOR_DOOR_1	22
ProtoDev.SENSOR_DOOR_2	23

1.2.3 Callback states

hatch status code	clarification
-1	open state
0	closed state

1	Open or close the execution in progress

IV. Common interface definitions

4.1 Callback methods

- IDataCallback is the data interface.

```
public interface IDataCallback {

  void success(String result).

  void error(ApiError error).
}
```

- The IProgressCallback interface indicates that this return progress can be used when processing files.

```
public interface IProgressCallback extends IDataCallback {

  void readyToSend(Request).

  void progress(int percent).
}
```

4.2 Interface data definitions

4.2.1 Return success

The "result" in the success method returns the data in json string format, as follows:

```
{
    "status": 0,
    "code": 0,
    "msg": "success",
    "data": {}
}
```

status	code	msg	data	note
0	0	success	json	Successful request, return data in json
1	-1	error	json	Request an error to return data in json.

4.2.2 Failure to return

Format of "ApiError" in error method

code	msg	note
19	Method Not Allowed	method is not supported
21	Empty Data	empty data

4.3. Interface invocation methods

```
PeanutSDK.getInstance().xxx()
```

4.4. motor interfaces

Get the status information of the motor and motor lock enable control.

```
PeanutSDK.getInstance().motor().xxx()
```

4.4.1 Getting the motor status

```
/**
 * :: Get status of motor.
 *
 * @param callBack the callback after request
 */
public void getStatus(IRobotCallBack callBack)


Return results
{
    "data": {
        "desc": "MOTOR_STATUS_LOCKED".
        "status": 255
    }, }
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.4.2 Setting the motor to be enabled or disabled

```
/**
 * Set motor enable.
 *
 * @param callBack the callback after request
 * @param enable on or off
 */
public void enable(IRobotCallBack callBack, int enable)

Return results
{
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.4.3. Setting HRC to be enabled or disabled

```
/**
 * Set HRC enable.
 *
 * @param callBack the callback after request
 * @param enable on or off
 */
public void hrc(IRobotCallBack callBack, boolean enable)
```

```
Return results
{
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.4.4. Obtaining motor health status

```
/**
 * :: Get health status of motor.
 *
 * @param callBack the callback after request
 */
public void getHealth(IRobotCallBack callBack)

Return results
{
    "data": [
        {
            "code": 0,
            "desc": "",
            "position": 0
        }
    ],
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.5. Device Information Interface

Get information about the robot's hardware devices.

```
PeanutSDK.getInstance().device().xxx()
```

4.5.1. Getting a list of devices

```
/**
 * Get the device list.
 *
 * @param callBack the callback after request
 */
public void getList(IDataListener callBack)

Return results
{
    "data": [
        {
            "id": 1,
            "name": "robot-arm".
            "pid": 4
        }, }
        {
            "id": 2,
            "name": "robot-stm32".
            "pid": 0
        }, }
        {
            "id": 3.
```

```
            "name": "motor".
            "pid": 1736260526
        }, }
        {
            "id": 6.
            "name": "power",
            "pid": 1134105942
        }
    ],
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.5.2. Getting device version information

```
/**
 * :: Get the detail information of board.
 *
 * @param callBack the callback after request
 * @param board the board
 */
public void getBoardInfo(IRobotCallBack callBack, String board)

Return results
{
    "code": 0,
    "data": {
        "device": "robot-arm".
        "sn": "",
```

```
        "version": {
            "sw": {
                "number": "1.0.14beta",
                "base": "linux-armv7-robot-v1.0.14-gf4fbb90-release.zip",
                "md5": "9205b9033196ba81"
            }, }
            "hw": {
                "number": "1.0.0"
            }, }
            "env": {
                "number": "Linux firefly 3.10.0",
                "base": "linux-rk3288-os-update-20180410.img",
                "md5": "9d40e1329780d62e"
            }
        }
    }, }
    "msg": "success",
    "status": 0
}
```


4.5.3. rebooting equipment

```
/**
 * :: Reboot the robot.
 *
 * @param callBack the callback after request
 */
public void reboot(IRobotCallBack callBack)
```

Return results

```
{
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.5.4. Getting configuration properties

```
/**
 * :: Get the configuration of device.
 *
 * @param callBack the callback after request
 */
public void getConfig(IRobotCallBack callBack)

Return results
{
    "data": {
        "params": xxx //map<string, string>
    }, }
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.5.5. Updating configuration properties

```
/**
```

```
 * :: Set the configuration with params.
 *
 * @param callBack the callback after request
 * @param params the param
 */
public void updateConfig(IRobotCallBack callBack, String params)

Return results
{
    "data": {
        "errorStr": "robot.type2"
    }, }
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.5.6. Setting whether the emergency stop button is enabled or not

```
/**
 * Set the scram button enable.
 *
 * @param callBack the callback after request
 * @param enable on or off
 */
public void setButtonEnable(IRobotCallBack callBack, boolean enable)

Return results
{
```

```
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.5.7. Getting the status of the emergency stop button

```
/**
 * :: Get status of the scram button.
 *
 * @param callBack the callback after request
 */
public void getScramButtonStatus(IRobotCallBack callBack)

Return results
{
    "data": {
        "info": false
    }, }
    "code": 0,
    "msg": "success",
    "status": 0
}
```

4.7 Movement control interface

```
PeanutSDK.getInstance().schedule().xxx()
```

4.7.1 Obtaining Scheduling Status

```
/**
 * :: Get status of motor.
 *
 * @param callBack the callback after request
 */
public void getStatus(IRobotCallBack callBack)
```

1. Waiting

```
{
    "status": 0,
    "code": 0,
    "msg": "success",
    "data": {
        "status": 1.
        "msg": "stop",
        "data": {
            "local": {
                "id": 1,
                "x": 1.23,
                "y": 2.34
            }, }
            "other": {
                "id": 2,
                "status": 0,
                "x": 1.23,
                "y": 2.34,
                "path": [
                    {
                        "x": 1.23,
```

```
                    "y": 2.34
                }, }
                {
                    "x": 1.23,
                    "y": 2.34
                }, }
                {
                    "x": 1.23,
                    "y": 2.34
                }, }
                {
                    "x": 1.23,
                    "y": 2.34
                }
            ]
        }, }
        "distance": 5.23.
        "time": 20
    }
  }
}
```

field	instance value	effective value	clarification
status	1	0,1,2,3	The current state of the robot, suspended by scheduling
data-local-id	1		Current robot's scheduling ID
data-local-x	1.23		Current robot position x in meters
data-local-y	2.34		Current robot position y, in meters
data-other-id	1		Dispatch ID of another robot

data-other-status	0	0,1,2,3	Status of the other robot, normal motion
data-other-x	1.23		Position of the other robot x, in meters
data-other-y	2.34		Position of the other robot y, in meters
data-other-path			The route of the other robot
data-distance	5.23		Remaining distance of route conflict, in meters
data-time	20		Remaining time for route conflict, in seconds

2. Avoidance


```
{
    "status": 0,
    "code": 0,
    "msg": "success",
    "data": {
        "status": 2.
        "msg": "avoid",
        "data": {
            "local": {
                "id": 1,
                "x": 1.23,
                "y": 2.34
            }, }
            "other": {
                "id": 2,
                "status": 1.
                "x": 1.23,
                "y": 2.34,
                "path": [
```

```
                    {
                        "x": 1.23,
                        "y": 2.34
                    }, }
                    {
                        "x": 1.23,
                        "y": 2.34
                    }, }
                    {
                        "x": 1.23,
                        "y": 2.34
                    }, }
                    {
                        "x": 1.23,
                        "y": 2.34
                    }
                ]
            }, }
            "distance": 5.23.
            "time": 20
        }
    }
}
```

field	instance value	effective value	clarification
status	2	0,1,2,3	The current state of the robot, being dispatched to avoid the
data-local-id	1		Current robot's scheduling ID
data-local-x	1.23		Current robot position x in meters
data-local-y	2.34		Current robot position y, in meters

data-other-id	1		Dispatch ID of another robot
data-other-status	1	0,1,2,3	The status of the other robot, suspended by scheduling
data-other-x	1.23		Position of the other robot x, in meters
data-other-y	2.34		Position of the other robot y, in meters
data-other-path			The route of the other robot
data-distance	5.23		Remaining distance of route conflict, in meters
data-time	20		Remaining time for route conflict, in seconds

3. Origin

```
{
    "status": 0,
    "code": 0,
    "msg": "success",
    "data": {
        "status": 3.
        "msg": "origin",
        "data": {
            "local": {
                "id": 1,
                "x": 1.23,
                "y": 2.34
            }, }
            "from": {
                "id": 60001,
                "x": 1.23,
                "y": 2.34
            }, }
```

```
            "to": {
                "id": 60002.
                "x": 3.23.
                "y": 4.34
            }
        }
    }
}
```

field	instance value	effective value	
status	3	0,1,2,3	Current state of the robot, adjusting the origin
data-local-id	1		Current robot's scheduling ID
data-local-x	1.23		Current robot position x in meters
data-local-y	2.34		Current robot position y, in meters
data-from-id	60001		Current origin ID
data-from-x			Position of the current origin x, in meters
data-from-y			Position of the current origin y, in meters
data-to-id	60002		Adjusted origin ID
data-to-x			Adjusted position of the origin x, in meters
data-to-y			Adjusted position of the origin y, in meters

4.7.2 Obtaining scheduling online status

```
/**
 * :: Get status of motor.
 *
 * @param callBack the callback after request
 */
```

```
public void getLiveStatus(IRobotCallBack callBack)


{
    "status": 0,
    "code": 0,
    "msg": "success",
    "data": {
        "local": {
            "id": 1,
            "status": 1.
            "x": 1.23,
            "y": 2.34,
            "connection": [
                1.
                3.
                2.
                3.
                3
            ]
        }, }
        "other": [
            {
                "id": 2,
                "status": 0,
                "x": 1.23,
                "y": 2.34,
                "connection": [
                    1.
                    3.
                    2.
                    3.
                    3
```

```
        ]
    }, }
    {
        "id": 3.
        "status": 1.
        "x": 1.23,
        "y": 2.34,
        "connection": [
            1.
            3.
            2.
            3.
            3
        ]
    }, }
    {
        "id": 4.
        "status": 2.
        "x": 1.23,
        "y": 2.34,
        "connection": [
            1.
            3.
            2.
            3.
            3
        ]
    }, }
    {
        "id": 5.
        "status": 3.
        "x": 1.23,
```

```
                "y": 2.34,
                "connection": [
                    1.
                    3.
                    2.
                    3.
                    3
                ]
            }
        ]
    }
}
```

(numeric, data) field	instance value	effective value	clarification
other-y	2.34		Position of the robot y, in meters
other-x	1.23		Position of the robot x in meters
other-status	1	0,1,2,3	Scheduling status of the robot
other-id	2		Scheduling ID of the robot
local-y	2.34		Current robot position y, in meters
local-x	1.23		Current robot position x in meters
local-status	1	0,1,2,3	Current robot scheduling status
local-id	1		Current robot's scheduling ID
local-connection	1,3,2,3,3	0,1,2,3	0-no connection 3-strong signal; there are 5 numbers that indicate, in order, the strength of the connection with the robot numbered 1, and with numbered 2,...
local-connection	1,3,2,3,3	0,1,2,3	0-no connection 3-strong signal; there are 5 numbers that indicate, in order, the

			strength of the connection with the robot numbered 1, and with numbered 2,...

4.7.3 Movement control suspension

```
/**
 * :: Get status of motor.
 *
 * @param callBack the callback after request
 */
public void pause(IRobotCallBack callBack)
```

4.7.4 Scheduling recovery

```
/**
 * :: Get status of motor.
 *
 * @param callBack the callback after request
 */
public void resume(IRobotCallBack callBack)
```

4.9 Subscription interface

4.9.1. Starting a subscription to the interface

Specify the topic to create a subscription, the topic in the returned data is the subscribed topic.

```
/**
 * Subscribe the topic and return data in callback.
```

```
 *
 * @param topic the topic
 * @param callBack the callback after request
 */
public void subscribe(String topic, IRobotCallBack callBack)
```

4.9.2. unsubscribe interface

Unsubscribe from specified topics

```
/**
 * :: Cancel subscribed topic.
 *
 * @param topic the topic
 */
public void unSubscribe(String topic)
```

4.10 Subscription to the list of topics

thematic	descriptive
BATTERY_STATUS	power state
BUTTON_STATUS	Emergency stop button status
ELEVATOR_FORWARD	Elevator request
ELEVATOR_STATUS	Elevator Status
GUIDE_STATUS	guide point status
MOTOR_STATUS	Motor status
NAVIGATION_STATUS	navigation state
RUNTIME_HEALTH	state of health

RUNTIME_HEARTBEAT	cardiac state
RUNTIME_OBJECT	Moving Object Status
SENSOR_COLLISION	crashbar state
SCHEDULE_LIVE	Scheduling online status
SCHEDULE_STATUS	scheduling state
UPDATE_STATUS	Upgrade Status
VENDOR_MATCH	Vending Machine Matching Status
NAVIGATION_PATH	Reporting of location and route information

4.10.1 Querying the power status

```
TopicName.BATTERY_STATUS


Return results
{
    "data": {
        "event": 0,
        "power": 83
    }, }
    "code": 0,
    "msg": "success",
    "status": 0,
    "topic": "BatteryStatusApi"
}
```

4.10.2 Querying the emergency stop button status

```
TopicName.BUTTON_STATUS


Return results
{
    "data": {
        "info": false
    }, }
    "code": 0,
    "msg": "success",
    "status": 0,
    "topic": "ButtonStatusApi"
}
```

4.10.3. Querying the motor status

```
TopicName.MOTOR_STATUS


Return results
{
    "data": {
        "desc": "MOTOR_STATUS_LOCKED".
        "status": 255
    }, }
    "code": 0,
    "msg": "success",
    "status": 0,
    "topic": "MotorStatusApi"
}
```

4.10.4 Querying guide point information

```
TopicName.GUIDE_STATUS


Return results
{
}
```

4.10.5. Querying the navigation status

```
TopicName.NAVIGATION_STATUS


Return results
{
    "data": {
        "desc": "idle".
        "dst": 1,
        "remain_length": 50.238235,
        "remain_time": 100.476471,
        "schedule": 0,
        "status": 82.
        "total_length": 61.702114.
        "total_time": 123.404228
    }, }
    "code": 0,
    "msg": "success",
    "status": 0,
    "topic": "NavigationStatusApi"
```

```
}
```

4.10.6 Querying health status

```
TopicName.RUNTIME_HEALTH


Return results
{
    "data": [
        {
            "code": 513.
            "desc": "CODE_ERROR_GAZER_NO_DATA",
            "pose": {
                "phi": -7.47E-4.
                "x": 1.32E-4.
                "y": -4.916307E-8
            }
        }
    ],
    "code": 0,
    "msg": "success",
    "status": 0,
    "topic": "RuntimeHealthApi"
}
```

4.10.7. Querying Heartbeat Status

```
TopicName.RUNTIME_HEARTBEAT
```

Return results


```
{
    "data": {
        "stm32": {
            "desc": "stm32 online",
            "status": 16
        }, }
        "sync": {
            "desc": "init",
            "status": 1
        }, }
        "motor": {
            "status": 255.
            "desc": "MOTOR_STATUS_LOCK"
        }, }
        "mode": {
            "status": 16.
            "desc": "WORK_STATUS_AUTOMODE_IDLE"
        }, }
        "time": 414754,
        "timestamp": 14499885
    }, }
    "code": 0,
    "msg": "success",
    "status": 0,
    "topic": "RuntimeHeartbeatApi"
}
```


4.10.8 Querying information on moving objects

```
TopicName.RUNTIME_OBJECT


Return results
{
    "data": {
        "exist": false,
        "x": 0.0,
        "y": 0.0
    }, }
    "code": 0,
    "msg": "success",
    "status": 0,
    "topic": "RuntimeObjectApi"
}
```

4.10.9. Querying collision bar status

```
TopicName.SENSOR_COLLISION


Return results
{
    "data": {
        "back": true,
        "front": false,
        "left": false,
        "mid-left": false,
        "mid-right": false,
        "right": false,
        "state": 16
    }, }
```

```
    "code": 0,
    "msg": "success",
    "status": 0,
    "topic": "SensorCollisionApi"
}
```

4.10.10. Query upgrade status

```
TopicName.UPDATE_STATUS

Return results
{
}
```

V. Constant field values

5.1 Error codes

Error Information Class The application layer can use this class to obtain error description information, as well as specific error codes.

error code	clarification	prescription
203 000	Initialization parameter Context is empty	
203 001	Initialize Component to empty	

203 002	Initialization privilege authorization failed	
203 020	authorizations	
203 021	Authorization Verification Failure	
203 022	The authorization parameter is empty	
203 023	Authorization secret mismatch	
203 024	Authorization certificate error	
203 025	Authorized machine mismatch	
203 026	Authorization application mismatch	
203 027	Authorization AppId mismatch	
203 028	Authorization fingerprint information error	
203 029	Authorization time illegal	
203 030	Authorization Fingerprint Code Exception	
203 031	Authorized fingerprint decoding exception	
203 032	expired authorization	
203 040	Properties	
203 060	Other	

203 100	Serial	
203 101	Connection timed out, communication disconnected	
203 102	Connection restored. Communication normal.	
203 150		
203 151	Coap's onError	
203 152	Coap returns null data	
203 170	Coap	
203 171	Coap Request Error	
203 172	Coap request returns null	
203 173	Coap request timeout	
203 200	sockets	
203 300	power supply	
203 301	Go to Charge Timeout	
203 302	Target point does not exist	
203 303	Route does not exist	
203 304	Failed to match charging post Not matched once	
203 305	Match charging post timeout Match to at least once	

203 306	No tag data timeout	
203 307	No laser data timeout	
203 308	No stm32 data timeout	
203 309	Timeout for failing to enter charging status within 60 seconds of starting to match the charging post	
203 310	No 5V	
203 311	Accidental disconnection of charging	
203 312	Abandon charging beyond the maximum number of matches	
203 313	No fsm set	
203 314	Disconnect retry limit reached	
203 315	Failed to subscribe to power api	
203 316	5V detected, but no current timeout detected	
203 317	Failed to go to automatic charging	
203 350	electrical machinery	
203 400	navigator	
203 401	Route planning failed, target point does not exist	
203 402	Route planning failed. Route does not exist.	
203 403	Route planning failed, robot position lost	

203 404	Route planning fails, optimal algorithm exceeds limit by 5	
203 405	Autonomous navigation failed, no target point	
203 406	Autonomous navigation failed, no route	
203 407	Autonomous navigation failed, no route	
203 408	Autonomous navigation failed. Charging.	
203 409	Autonomous navigation fails, arrival strategy is empty	
203 410	Autonomous navigation failed, machine not idle	
203 411	Autonomous navigation failed beyond the set number of cycles	
203 550	file transfer	
203 551	transmission timeout	
203 552	Compression Failure	
203 553	Invalid compressed path	
203 554	Input file parameters are empty	
203 555	Robot failed to verify md5.	
203 557	Failed to get uploaded file	
203 558	The zip is empty	
203 559	Failed to get upload information	
203 560	Invalid unzip file	

203 561	APP Failed to check md5	
200 500	hatch	
200 501	Failed to get hatch type/unknown	
200 502	Illegal or non-existent hatch ID	
200 503	Failed to get the status of the corresponding hatch	

5.2 Events - Equipment Operation

numerical value	Description of the incident	Data types and descriptions
10000	Failure to execute instruction	null
10001	Configuration update reboot startup	String, text description
10002	Configuration update reboot to start initialization	String, text description
10003	Configuration update reboot is ready	String, text description
10004	Configuration update failure	String, text description, update failed configuration item
10005	Configuration update successful	
10006	Pinging successfully.	
10007	Ping Failure	

10010	Changes in working modes	int, operating mode value
10011	Electricity change	int Electricity value
10012	Robot IP changes	String, IP Address
10013	Emergency button status change	boolean, true: on, false: off
10014	Motor lock status change	int, motor status
10015	Change in total mileage	double, total mileage
10016	Positioning for success	
10017	Robot synchronized time successfully.	
10018	Obtaining location and route information was successful	Json String

5.3 Events - Power Charge Events

numerical value	Description of the incident	Data types and descriptions
40001	Arriving at the charging station	int
40002	Failed to match charging post Not matched once	int

40003	Match charging post timeout Match to at least once	int
40004	No tags. Timeout.	int
40005	No laser. Time out.	int
40006	No stm32 timeout	int
40007	Timeout for failure to enter charging status within 60 seconds from the start of the charging post matching timer	int
40008	No 5v	int
40009	5V detected, but no current timeout detected	int
40010	accidental disconnection	int
40011	Charging.	int
40012	Abandon charging beyond the maximum number of matches	int
40013	Re-match the charging post to the charging point (target point)	int
40014	Re-match the charging post to the charging point	int
40015	undefined	int
40016	undefined	int

40017	undefined	int
40018	undefined	int
40019	undefined	int
40020	Exit Charge	int
40021	undefined	int
40022	undefined	int
40023	Charge Timeout	int

5.4 Constants - Operating Modes

Defines the working modes that can be switched by the robot. Belongs to the ApiConstants class.

constant field	clarification
AUTO	default (setting)
BUILD_MAP	schema construction mode
MFG_TEST	test pattern
CHARGER	charging mode
MANUAL	manual mode
APAPTER_CHARGER	Line charging mode
SELF_TEST	autotest mode

5.5 Constants - Navigation Status

---

defines the state of the robot during autonomous navigation.

numerical value	clarification
0	leisure time
1	Ready. Route planning.
2	on the march
3	arrived
4	pause (media player)
5	collisions
6	impede
7	cessation
8	malfunctions
9	in the block
10	close

5.6 Constants - Route Planning Strategies

Defines a strategy for robot route planning. Belongs to the PeanutNavigation class.

constant field	clarification
POLICY_ADAPTIVE	optimal route
POLICY_FIXED	Fixed route, target point sequence

5.7 Constants - Manual Navigation Directions

Defines the direction of motion when the robot is manually controlled. Belongs to the Navigation class.

constant field	clarification
FORWARD	advance
BACKWARD	be back
LEFT	turn right
RIGHT	turn right

5.8 Constants - Power Status

numerical value	clarification
1	leisure time
2	Automatically going to charge.
3	Manually going to charge in
4	Charging
5	Adapter charging
6	Charging status is being canceled

5.9 Constant - Power Charge Command

Defines the robot trigger charging command. Belongs to the PeanutCharger class.

constant field	clarification
CHARGE_ACTION_AUTO	automatic charging
CHARGE_ACTION_MANUAL	Manual charging
CHARGE_ACTION_ADAPTER	Adapter Charging
CHARGE_ACTION_STOP	Stop charging

5.10 Constants - Motor Status

Defines the state of the motor.

numerical value	clarification
16	Push Button Unlock
32	app unlock
49	Right motor communication error
50	Left motor communication error
51	Right motor internal error
52	Left motor internal error
255	lock (denying access to a computer system or device or files)

六、Update Log

V1.3.0

June 18, 2024

- Added T3 hatch control

V1.2.9

May 10, 2024

- Added Ros map file import/export function.

V1.2.7-1

November 2, 2023

- Optimize the initialization logic in the demo
- Save last initialization SDK type

V1.2.7

April 15, 2023

- Added T8 Dinner Plate Light Demo

V1.2.6

November 24, 2022

- Completion of 5.1 error code

V1.2.5

September 2, 2022

- Navigation page rewrite adds Listen

V1.2.4

August 15, 2022

- New Audio Component

V1.2.3

May 1, 2022

- Simplify document content
- Remove components such as sanitization, file transfer, etc.

V1.2.2

January 18, 2022

- Optimizing navigation timing issues
- Whole machine communication optimization
- Modify the SDK initialization method
- Modify the build.gradle dependency, see Sample for details

V1.0.1

December 2, 2021

- Optimize offline authentication
- Whole machine communication optimization
- New method to get real-time location information
- New runtime component events
- Added real-time location information subscription thread

V1.0.0

October 13, 2021

- Added offline authentication
- Whole machine communication optimization
- Added Map Transfer component
- Added Gravity Sensor Component
- Update power supply assembly (new status)

VII. FAQ

common problems

Android system interface without bottom menu bar, can't be operated

Click [Settings], [Display], uncheck the box to hide the status bar

Cannot get data properly, control machine failed, can control expression, hatch, etc., regular model

Check if there is a food delivery app running on Android, if so force stop quit the app and retry again

Cannot get data properly, control machine failed, can control expression, hatch, etc., G1, W3 models

For pure laser models, you need to set the communication protocol to `REMOTE_LINK_PROXY`, see SDK initialization for details.